

RAG PDF Research Corpus System – LangGraph + GPT-5.1 Thinking

Version: 2.1

Target environment: Google Colab (ipynb), Google Drive, OpenAI API, LangGraph 1.x

0. Purpose & Scope

Build a **Google Colab–based notebook** that orchestrates a LangGraph workflow using **GPT-5.1 Thinking** and advanced

1. **Ingest PDFs** from a user-specified **Google Drive folder**.
2. **Parse and chunk** each PDF into meaningful text segments.
3. **Extract metadata** (title, authors, publish date, venue, IDs) per paper.
4. **Generate high-quality summaries** and agent notes for each paper.
5. **Embed and index** all chunks in a FAISS vector store for RAG.
6. **Derive a three-tier topic taxonomy** (broad → mid → fine) from the corpus using clustering + GPT-5.1.
7. **Classify each paper** into one topic at each tier, with confidence scores and reasoning notes.
8. **Persist results** into:
 - A **master CSV** (and/or Parquet) "database" of all papers.
 - A **FAISS index** and metadata mapping.
 - A **taxonomy JSON** with versioning.
9. Provide **basic QC and RAG query tools** for exploration.

The system is designed for **AI/ML/LLM research PDFs** (often from arXiv), but should be robust to other technical paper

Non-goals (for now):

- Building a full web UI (Gradio/Streamlit) – this is Colab-focused.
- Fine-tuning models or running fully local LLMs in this notebook.

1. Environment, Dependencies, and Runtime Assumptions

1.1 Runtime

- **Google Colab** notebook (`.ipynb`).
- User has:
 - OpenAI API key.
 - A Google Drive folder with PDFs.

1.2 Core Libraries

Install (via `pip`) at top of notebook:

- **OpenAI Python SDK** (latest – supports GPT-5.1 and embeddings).
- **LangGraph** (Python; workflows + durable checkpoints).
- **PyMuPDF** (`.fitz`) for PDF parsing.
- **FAISS** (CPU) for vector indexing.
- **scikit-learn** for clustering (e.g., Agglomerative / KMeans).
- **hdbscan** (optional) for density-based clustering.
- **pandas**, **numpy**, **tqdm**, **matplotlib** for data & visualization.
- **python-dateutil** for date parsing.
- **requests** for arXiv / CrossRef API access.

- **pytesseract** and **Pillow** (optional) for OCR fallback.

2. High-Level Architecture

2.1 Architectural Style

- Orchestrated via **LangGraph** in a **supervisor-worker pattern**:
 - **Supervisor** state tracks global config, list of papers, taxonomy, progress.
 - **Workers** operate on a single `paper_id` at a time (parse, embed, summarize, classify).
- Multi-stage pipeline:
 1. **Ingestion Workflow** – parse, chunk, embed, metadata, initial summary, initial CSV.
 2. **Taxonomy Workflow** – cluster paper embeddings, build 3-tier taxonomy.
 3. **Classification Workflow** – assign final topics per tier with GPT-5.1.

2.2 Main Stages

1. Setup, config & API keys.
2. Mount Google Drive & discover PDFs.
3. PDF parsing, section-aware chunking & OCR fallback.
4. Embedding and FAISS index creation.
5. Pass 1 – metadata + summary + initial notes.
6. Initial master CSV export (with status tracking).
7. Topic modeling & taxonomy construction (Tier1/2/3).
8. Taxonomy review & approval (human-in-the-loop step).
9. Pass 3 – final topic classification & classification notes.
10. Final CSV export (with taxonomy version) + QC & RAG query utilities.

3. Data Model & State Schemas

3.1 `RunConfig`

Pydantic model or TypedDict initialized by user in a config cell:

- `drive_folder_path: str` – Google Drive path to folder with PDFs.
- `max_papers_per_run: int | None` – cap to limit cost in a run.
- `max_pages_per_paper: int | None` – optional page cap.
- `max_chunks_per_paper: int` – upper bound on chunks per paper.
- `enable_ocr_fallback: bool` – whether to try OCR on low-text PDFs.
- `summary_model: str` – e.g., `"gpt-5.1-thinking"`.
- `taxonomy_model: str` – typically same as summary model.
- `classification_model: str` – typically same as summary model.
- `use_tiered_models: bool` – if `True`, allow cheaper models for bulk tasks later.
- `summary_reasoning_effort: Literal["none", "low", "medium", "high"]`.
- `taxonomy_reasoning_effort: Literal[...]`.
- `classification_reasoning_effort: Literal[...]`.
- `embedding_model: str` – e.g., `"text-embedding-3-large"`.

- `cluster\_tier1\_target\_k: int | None` – optional target cluster count.
- `cluster\_tier2\_target\_k: int | None`.
- `cluster\_tier3\_target\_k: int | None`.
- `enable\_deep\_analysis\_pass: bool` – optional Pass 2 (deep methods/results summaries).
- `taxonomy\_approval\_required: bool` – if `True`, classification waits for user approval.
- Cost safety toggles:
 - `max\_tokens\_per\_summary: int`.
 - `max\_tokens\_per\_classification: int`.

3.2 `PaperRecord`

Represent each paper as a Pydantic model / TypedDict:

- `id: str` – internal ID (e.g., hash of `file\_path`).
- `file\_path: str` – absolute path in Colab.
- `filename: str`.
- `source: Literal["arxiv","doi","other"] | None`.
- `arxiv\_id: str | None`.
- `doi: str | None`.
- `title: str | None`.
- `authors: list[str] | None`.
- `venue: str | None`.
- `publish\_date: date | None`.
- `publish\_date\_source: Literal["arxiv","crossref","pdf","manual","unknown"]`.
- `is\_preprint: bool | None`.
- `arxiv\_version: str | None`.
- `raw\_text\_stats: dict` – e.g.:
 - `pages: int`.
 - `chars\_total: int`.
 - `chars\_per\_page: float`.
 - `alnum\_ratio: float`.
 - `parse\_quality\_score: float` (0–1).
- `abstract\_text: str | None` – cleaned/normalized abstract.
- `full\_summary: str | None` – high-level summary.
- `deep\_summary: str | None` – optional deeper pass summary.
- `initial\_notes: str | None` – notes from Pass 1 agent.
- `classification\_notes: str | None` – reasoning behind topic assignment.
- `tier1\_topic: str | None`.
- `tier2\_topic: str | None`.
- `tier3\_topic: str | None`.
- `tier1\_confidence: float | None`.
- `tier2\_confidence: float | None`.
- `tier3\_confidence: float | None`.

- `taxonomy\_version: str | None`.
- `processing_status: Literal[
"pending",
"parsed",
"summarized",
"embedded",
"deep_analyzed",
"classified",
"failed"
]`.
- `error\_reason: str | None` – error message or notes if `failed`.
- `last\_updated: datetime | None` – timestamp of last update.

3.3 `PaperChunk`

Chunk-level representation (for RAG & analysis):

- `paper\_id: str`.
- `chunk\_id: str`.
- `section\_label: str` – e.g., `abstract`, `introduction`, `methods`, `results`, `conclusion`, `other`.
- `page\_start: int`.
- `page\_end: int`.
- `text: str`.
- `embedding\_id: int | None` – index into FAISS index.

Vector values themselves are not stored here; they live in FAISS.

3.4 `TopicHierarchy`

JSON-serializable structure representing the 3-tier taxonomy:

```
{
  "taxonomy_version": "v1.0",
  "created_at_iso": "2025-11-18T12:00:00Z",
  "notes": "Initial taxonomy for corpus of N papers",
  "tiers": {
    "tier1": [
      {
        "id": "T1_LLMs",
        "label": "LLMs",
        "description": "Large Language Models, architectures, and training.",
        "paper_ids": ["p1", "p5", "p9"],
        "embedding": []
      }
    ],
    "tier2": [
      {
        "id": "T2_LLMs_Attention",
        "parent_tier1_id": "T1_LLMs",
        "label": "Transformer Attention",
        "description": "Attention mechanisms, efficient attention, and variants.",
        "paper_ids": ["p1", "p2"]
      }
    ]
  }
}
```

```

    }
  ],
  "tier3": [
    {
      "id": "T3_Attn_Efficient",
      "parent_tier2_id": "T2_LLMs_Attention",
      "label": "Efficient Attention Methods",
      "description": "Sparse, low-rank, kernelized, and approximate attention methods.",
      "paper_ids": ["p1"]
    }
  ]
}
}
}

```

3.5 Supervisor State (`GraphState`)

LangGraph global state object:

- `config: RunConfig`.
- `papers: dict[str, PaperRecord]` – keyed by `paper\_id`.
- `chunks: dict[str, list[PaperChunk]]` – per paper.
- `topic\_hierarchy: TopicHierarchy | None`.
- `taxonomy\_approved: bool` – set via notebook flag or user action.
- `faiss\_index\_path: str | None`.
- `faiss\_meta\_path: str | None`.
- `master\_csv\_path: str | None`.
- `errors\_log\_path: str | None`.

(Truncated in text for brevity - the actual implementation should keep the full spec.)